

Hướng dẫn ArgoCD trên Minikube

<https://gemini.google.com/app/a7c1ef3d5c4d2943>

User prompt: Hướng dẫn làm demo test dùng minikube với argocd để học argocd

Response: Dưới đây là hướng dẫn từng bước (step-by-step) để bạn dựng môi trường Lab học ArgoCD với **Minikube** ngay trên máy local theo mô hình **GitOps**.

Điều kiện tiên quyết (Prerequisites)

Cần cài đặt sẵn trên máy:

- **Docker Desktop** (hoặc Podman)
- **Minikube**
- **kubectl**
- **Git** và một tài khoản GitHub

Bước 1: Khởi tạo Cluster Minikube

Mở Terminal / Command Prompt và chạy lệnh:

```
# Khởi chạy minikube với tài nguyên đủ dùng cho ArgoCD
minikube start --cpus=4 --memory=8192 --driver=docker
```

```
# Kiểm tra cluster đã sẵn sàng chưa
kubectl get nodes
```

Cách xác nhận thành công: Trạng thái Node chuyển sang Ready.

Bước 2: Cài đặt ArgoCD vào Cluster

Tạo namespace riêng cho ArgoCD và áp dụng manifest chính thức:

```
# 1. Tạo namespace
kubectl create namespace argocd

# 2. Cài đặt ArgoCD
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml

# 3. Theo dõi các Pods đến khi tất cả chuyển sang trạng thái Running
kubectl get pods -n argocd -w
```

Cách xác nhận thành công: Các pod như argocd-server, argocd-repo-server, argocd-application-controller đều báo 1/1 Running.

Bước 3: Truy cập ArgoCD UI

1. Lấy mật khẩu admin ban đầu

Mật khẩu được ArgoCD tự động tạo và lưu trong Secret argocd-initial-admin-secret:

- Trên Linux / macOS:

```
kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.password}" | base64 -d; e
```
- Trên Windows (PowerShell):

```
[System.Text.Encoding]::UTF8.GetString([System.Convert]::FromBase64String((kubectl -n argocd get secre
```

2. Forward Port để truy cập giao diện Web

```
kubectl port-forward svc/argocd-server -n argocd 8080:443
```

- Mở trình duyệt truy cập: <https://localhost:8080> (bỏ qua cảnh báo SSL/Self-signed cert).
- **Username:** admin
- **Password:** Chuỗi vừa lấy ở bước trên.

Bước 4: Chuẩn bị Git Repository cho GitOps

Tạo một Repository trên GitHub (ví dụ: argocd-demo-apps) và đẩy cấu hình Kubernetes mẫu bên dưới vào.

Tạo file deployment.yaml:

```
apiVersion: apps/v1
kind: Deployment
metadata:
```

```

name: my-demo-app
namespace: default
spec:
  replicas: 2
  selector:
    matchLabels:
      app: my-demo-app
  template:
    metadata:
      labels:
        app: my-demo-app
    spec:
      containers:
      - name: web
        image: nginxdemos/hello:plain-text
        ports:
        - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: my-demo-service
  namespace: default
spec:
  type: ClusterIP
  ports:
  - port: 80
    targetPort: 80
  selector:
    app: my-demo-app

```

Commit và push file này lên nhánh main của Repo GitHub.

Bước 5: Tạo Application trên ArgoCD

Bạn có thể tạo bằng giao diện GUI hoặc dùng file Declarative Manifest. Dưới đây là cách khai báo bằng file K8s Manifest đúng chuẩn GitOps.

Tạo file argo-app.yaml:

```

apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: demo-app
  namespace: argocd
spec:
  project: default
  source:
    repoURL: 'https://github.com/USERNAME/argocd-demo-apps.git' # Thay URL repo GitHub của bạn
    targetRevision: HEAD
    path: '.' # Thư mục chứa file deployment.yaml
  destination:
    server: 'https://kubernetes.default.svc'
    namespace: default
  syncPolicy:
    automated:
      prune: true
      selfHeal: true

```

Áp dụng file này:

```
kubectl apply -f argo-app.yaml
```

Bước 6: Kiểm tra và trải nghiệm GitOps cơ chế Auto-Sync

- Mở UI ArgoCD (<https://localhost:8080>), bạn sẽ thấy ứng dụng demo-app hiển thị sơ đồ cây với các tài nguyên Deployment, ReplicaSet, Pod, Service với trạng thái **Synced** và **Healthy**.
- Kiểm tra tài nguyên thực tế dưới cluster:


```
kubectl get pods -l app=my-demo-app
```
- Thử nghiệm tính năng GitOps (Self-healing & Sync):
 - Sửa file deployment.yaml trên GitHub (ví dụ đổi replicas: 2 thành replicas: 4) rồi commit. ArgoCD sẽ tự phát hiện thay đổi và kéo 4 Pods về cluster.
 - Xóa thủ công 1 Pod bên dưới cluster bằng `kubectl delete pod <pod-name>`. ArgoCD/Kubernetes sẽ lập tức tự khởi tạo lại Pod mới để duy trì trạng thái mong muốn.

Dọn dẹp môi trường (Cleanup)

Sau khi hoàn thành bài Lab:

```
# Xóa minikube cluster
minikube delete
```

User prompt: Unable to load data: revision HEAD must be resolved

Response: Lỗi **Unable to load data: revision HEAD must be resolved** xảy ra khi ArgoCD không thể tìm thấy hoặc resolve được branch/commit mặc định (HEAD) trên Git Repository mà bạn đã khai báo trong Application.

Dưới đây là các nguyên nhân phổ biến nhất và cách xử lý nhanh:

Nguyên nhân 1: Repository còn Trống (Chưa có Commit nào)

Nếu bạn vừa khởi tạo một Repo trên GitHub/GitLab nhưng chưa commit file nào lên, Git Server chưa hề có branch main hay master, dẫn đến HEAD không trở vào đầu cả.

- **Cách xử lý:** Tạo ít nhất 1 file (ví dụ README.md hoặc file deployment.yaml) rồi push lên repo:

```
git add .
git commit -m "initial commit"
git push origin main
```

Nguyên nhân 2: Khai báo targetRevision: HEAD trên Repo không dùng HEAD

Trong file Application, thuộc tính targetRevision để là HEAD thường hoạt động tốt. Tuy nhiên, một số Git Server hoặc cấu hình mã nguồn mở yêu cầu chỉ định rõ tên branch.

- **Cách xử lý:** Mở file manifest Application (hoặc sửa trực tiếp trên giao diện ArgoCD UI phần **EDIT -> Target Revision**) và thay HEAD bằng tên branch cụ thể (ví dụ main hoặc master):

```
spec:
  source:
    repoURL: 'https://github.com/USERNAME/argocd-demo-apps.git'
    targetRevision: main # <--- Thay HEAD bằng main hoặc master
    path: '.'
```

Nguyên nhân 3: Sai URL Repo hoặc Repo Riêng Tư (Private Repo) chưa Authenticate

Nếu URL Repo bị gõ sai, hoặc Repo ở chế độ **Private** làm cho ArgoCD không thể kết nối tới để đọc branch/HEAD.

- **Cách xử lý:**

1. Thử truy cập URL repo bằng Trình duyệt web / git clone trên máy cá nhân xem URL có đúng không.
2. Nếu là Private Repo, bạn phải kết nối Credential vào ArgoCD trước bằng CLI hoặc UI:

- **Qua UI:** Vào **Settings -> Repositories -> Connect Repo** (điền HTTPS Token hoặc SSH Key).
- **Qua CLI:**

```
argocd repo add https://github.com/USERNAME/argocd-demo-apps.git --username <username> --passv
```

Nguyên nhân 4: Cache của ArgoCD Repo Server bị lỗi

Đôi khi ArgoCD lưu cache danh sách revision của Git repository và bị lệch trạng thái.

- **Cách xử lý:** Làm mới cache (Hard Refresh) ứng dụng trên ArgoCD UI:
 - Mở App trên ArgoCD UI -> Click nút xổ xuống bên cạnh **Refresh** -> Chọn **Hard Refresh**.
 - Hoặc restart lại Pod argocd-repo-server:

```
kubectl rollout restart deployment argocd-repo-server -n argocd
```

Kiểm tra nhanh kết quả

Sau khi thực hiện một trong các cách trên, bấm **Refresh** lại ứng dụng trên ArgoCD UI. Nếu trạng thái chuyển sang màu xanh **Synced / Healthy** hoặc bắt đầu hiện cây tài nguyên là bạn đã khắc phục thành công.